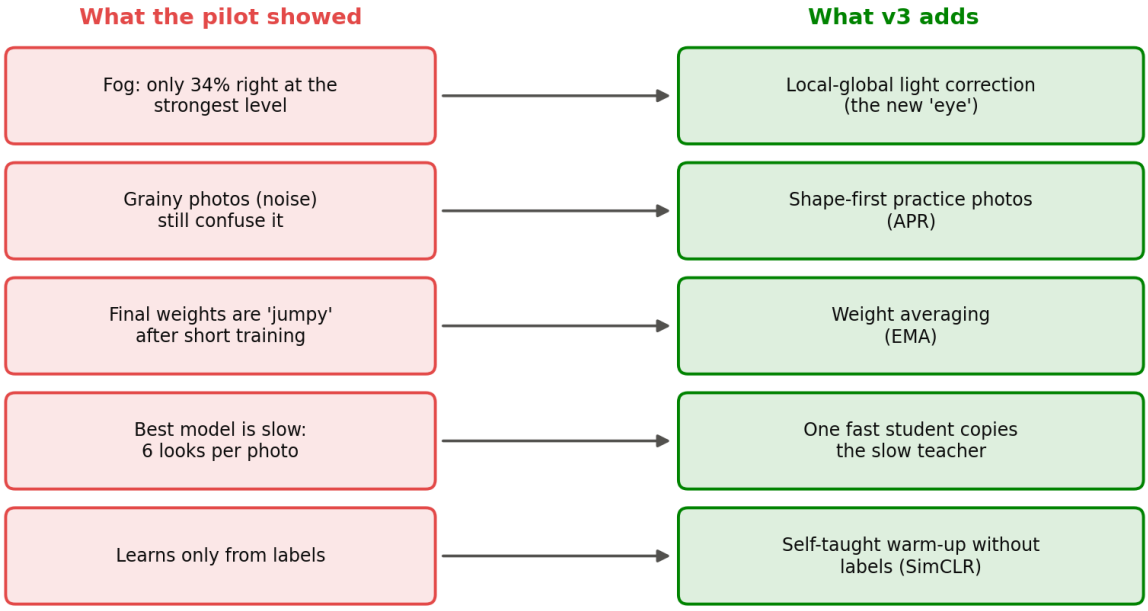
# RoadLens v3, explained simply

What is new in the third version of the traffic-sign reader, why each idea was added, and how to run it. No maths needed.

> Not tested yet: every idea here is written and checked for mistakes in the code, but none has been trained on a GPU. Whether it helps is only known after the next Colab run.

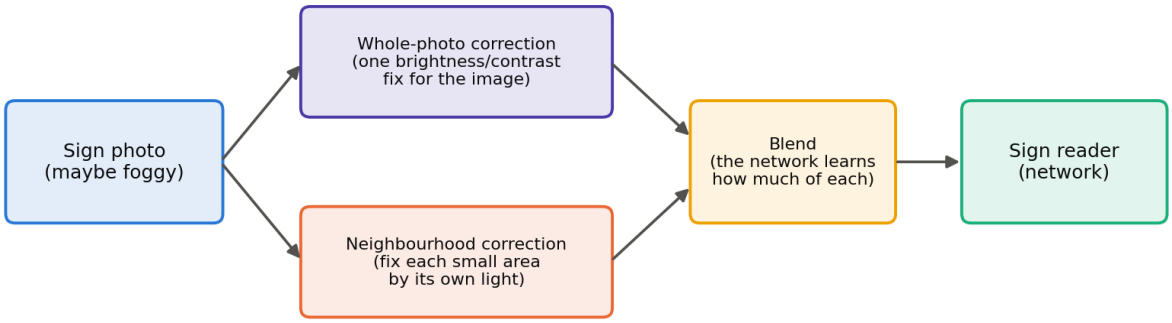
## Where we stand

The 15 October pilot showed that the v2 sign reader is excellent on clean photos (99%) and good on damaged ones (92%). But fog at its strongest is right only 34% of the time, grainy photos still cause mistakes, and the best model has to look at each photo six times, which is slow. v3 answers each of these with one idea.

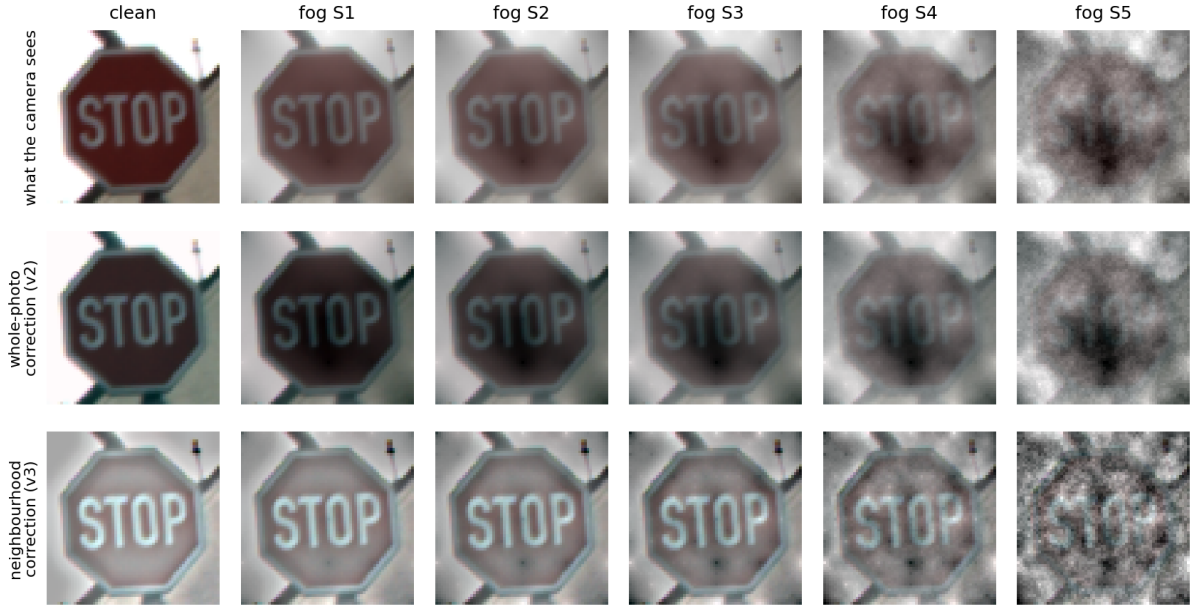


# Idea 1: a smarter 'eye' for fog

Fog does not darken a photo evenly: it lays a soft, uneven veil over it. v2 corrected the light of the whole photo at once, which cannot remove an uneven veil (fog stayed at about 68%). The v3 front end also corrects every small area by the light around it, and lets the network learn how much of each correction to use, so the sign's colour is not lost.




*Like your eyes adapting to light separately in the bright and the shaded parts of a scene.*




A real test photo with the project's own fog at every level (S1 light ... S5 strongest). Bottom row: what the v3 correction gives the network.

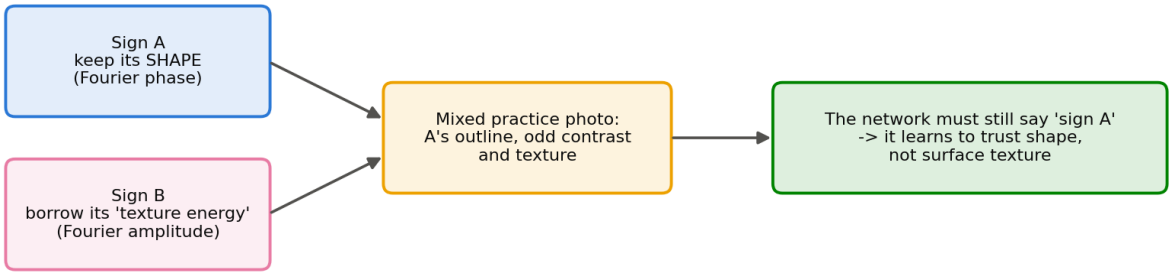
• **Fog S1-S3:** after the neighbourhood correction the foggy sign looks almost like the clean one. This is where the new eye should help.

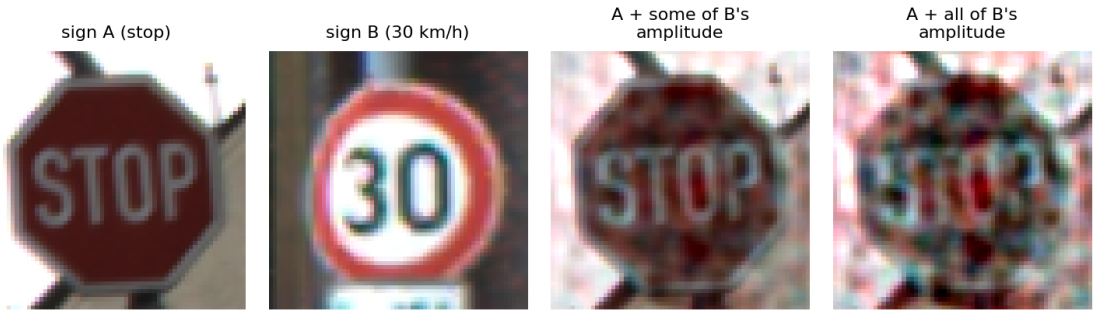
• **Fog S4:** it partly works.

• **Fog S5:** the sign has almost disappeared from the 64 x 64 photo; the correction mostly brings out the fog's own pattern. No front end can recover what is not there, so v3 should not be expected to fix fog at S5. That limit is itself a useful finding for the report.

# Idea 2: practise on 'shape-first' photos (APR)

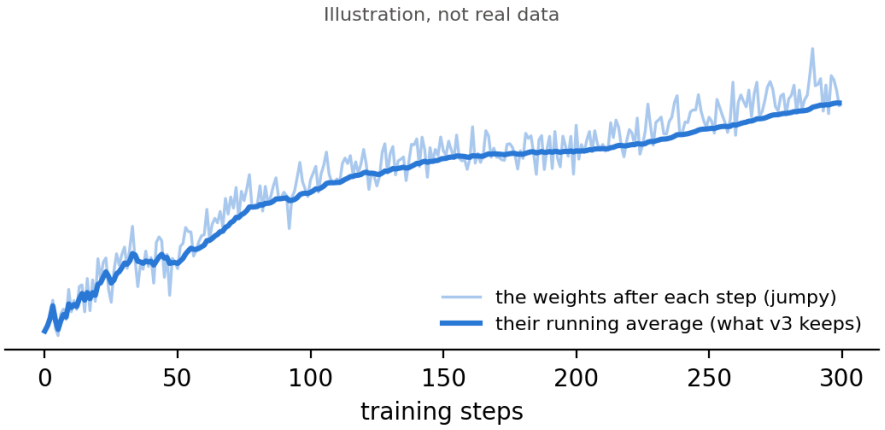
Every photo can be split into two ingredients: its **shape** (where the edges and symbols are) and its **texture energy** (how strong the contrast and fine grain are). Noise and fog mostly spoil the second. v3 makes practice photos that keep one sign's shape but borrow another sign's texture energy, and asks the network to still name the first sign. It learns to trust shape.




Real example of the new practice change. With some of B's texture energy the stop sign is clearly readable; with all of it, 'STOP' is only faintly visible, which is a hard practice case. Training mostly uses the milder mixes. No noise or blur is added, so this is not one of the 14 exam damages.

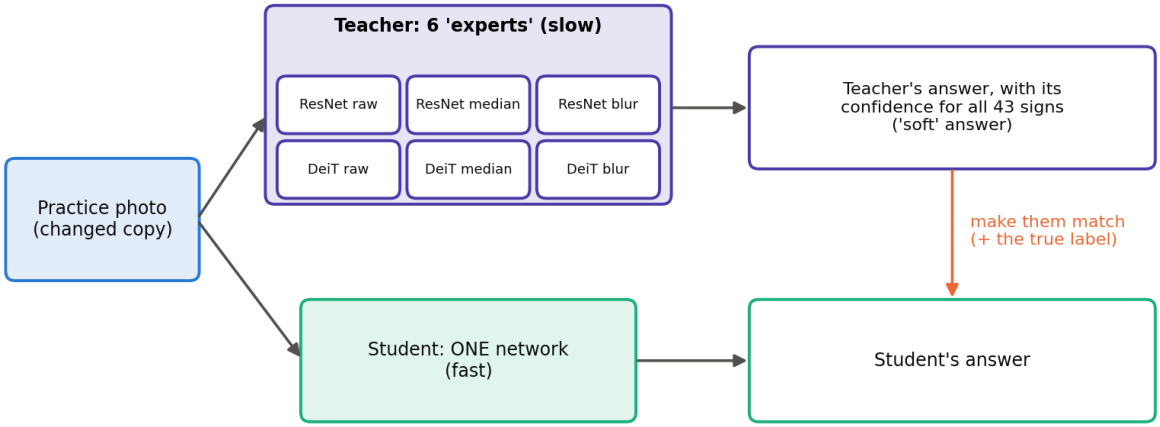
## Idea 3: keep the average, not the last step (EMA)




During training the network's settings jump around a little after every step. v3 keeps a running average of them and uses that average. It is free, and usually generalises better.

# Idea 4: a fast student copies the slow teacher

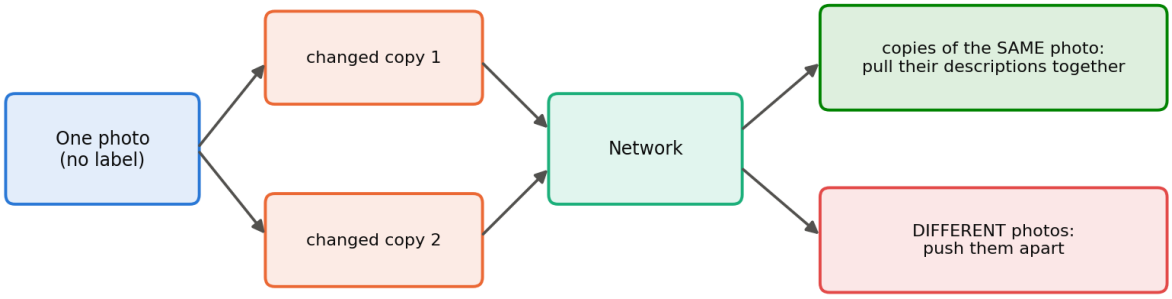
The best v2 model asks six 'experts' (two networks, each looking at three versions of the photo) and averages them. That is accurate but six times slower. v3 trains **one** network to give the same answers as the six experts, on exactly the same practice photos. Then only the student is used in the car: one look per photo.




• Two students are trained: a ResNet-18 and a small MobileNetV3 (made for phones and embedded chips).

• The teacher passes on more than the right answer: it also shows which other signs look similar, which is extra knowledge for the student.

## Idea 5: learn without labels first (SimCLR)




*Afterwards the network is fine-tuned with labels as usual (run resnet18_v3_ssl).*

Before seeing any labels, the network plays a game: two changed copies of the same photo must get similar descriptions, and different photos different ones. This teaches it to ignore the kind of change, which is exactly what robustness needs. This is the **unsupervised** part of v3.

# Other network designs tried

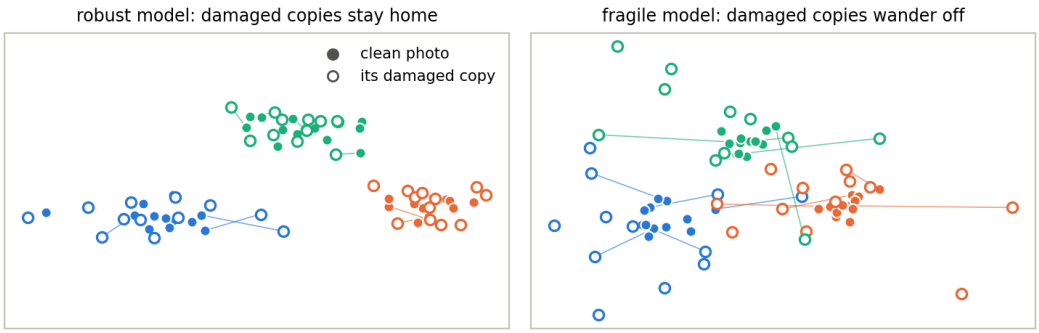
| Network | In simple words | Size | Why try it |
|---|---|---|---|
| ResNet-18 v3 | the v2 workhorse with ideas 1-3 | 11.2 M | the main comparison |
| ResNet-18 v3 at 64 px | same, but without enlarging the photo first | 11.2 M | about 4x less work per photo: is it still accurate? |
| ConvNeXt-Atto | a modern CNN with large filters and layer-norm | 3.4 M | known to be more robust than ResNets |
| EfficientNet-B0 | a CNN tuned for accuracy per computation | 4.1 M | strong and efficient |
| MobileNetV3-Large | a CNN built for phones | 4.3 M | fast deployment |
| DeiT-S v3 | the transformer with ideas 1-3 | 21.6 M | partner for the v3 ensemble |

Parameter counts with the 43-sign output layer. All use the same v3 training, so only the design differs.

## Other learning algorithms tried (Experiment 9)

| Algorithm | How it works | Kind |
|---|---|---|
| HOG + linear SVM | hand-made edge-direction features, then the best separating lines | classical, supervised |
| HOG + random forest | the same features, then a vote of 200 decision trees | classical, supervised |
| HOG + PCA + kNN | compress the features, then copy the label of the 5 most similar training photos | classical, supervised |
| SimCLR features + kNN / logistic regression | features learned without labels, then a simple classifier | self-supervised |
| k-means stability | group photos without labels; check whether damaged copies stay in their group | unsupervised analysis |

Illustration: each colour is one group found by k-means without labels



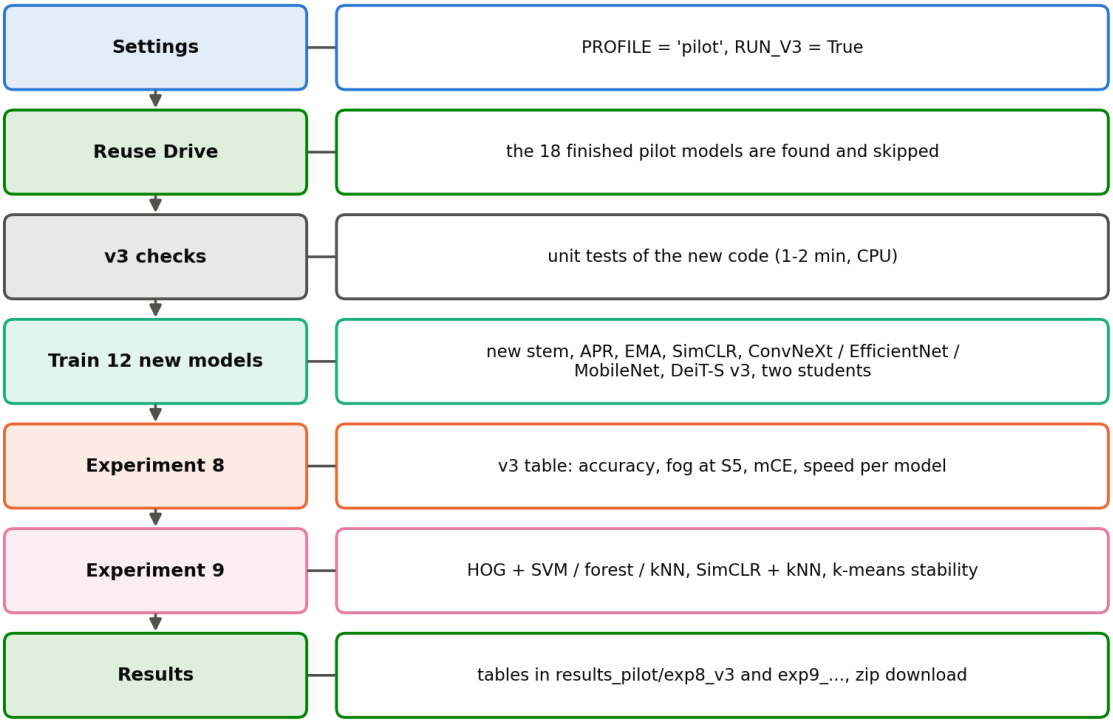

The k-means test gives a robustness score **without using any labels**: the share of damaged photos that land in the same group as their clean original.

# Faster training on the T4

| Speed-up | In simple words |
|---|---|
| channels-last memory layout | stores images in the order the GPU's fast units like best |
| fused AdamW | updates all settings in one GPU call instead of many |
| cuDNN autotuning | the GPU tries several ways to compute and keeps the fastest |
| native 64 px option | skips enlarging the 64 px photo to 128 px: about 4x fewer pixels |
| one-network student | at test time 1 look per photo instead of 6 |
| torch.compile (optional, off) | turns the network into faster GPU code; slow to start, so off by default |

Experiment 8 measures the real speed of every model on the GPU (photos per second, and milliseconds per single photo), so speed and accuracy can be compared side by side.

## How to run it




• Upload the new **RoadLens_standalone.ipynb** to Colab, choose a T4 GPU, keep PROFILE = 'pilot' and RUN_V3 = True, and press Run all. It reuses your finished models on Drive.

• Rough time for the new part: 1.5-2.5 hours of GPU (an estimate, not measured).

# How to read the results fairly

• **Choose with the mock exam, not the final exam.** The v3 ideas were designed after looking at which damages the pilot failed (fog, noise), so the final damaged exam is no longer a surprise for v3. The fair column is 'Val-C (sel.)': four damages that are never part of the final exam.

• **One seed, 2,000 photos.** Differences under about one point can be luck; the full 3-seed run decides.

• **APR changes contrast.** It adds no noise or blur, but contrast and fog are also contrast changes, so a gain on those two needs care.

• **Expectations, not results:** the local correction is aimed at fog, APR at noise, the student at speed. Any of them may turn out not to help; the tables will show which.

## Where the new code lives

| Idea | Setting in the run files | Code |
|---|---|---|
| local-global correction | model.input_norm: local | roadlens/models/base.py |
| APR practice photos | augment.ops: [..., apr] | roadlens/augment/ops.py |
| weight averaging | train.ema | roadlens/engine.py |
| student / teacher | train.distill | roadlens/engine.py |
| self-taught warm-up | train.objective: simclr, model.init_from | roadlens/ssl.py |
| experiments | configs/experiments/exp8_v3.yaml, exp9_... | scripts/exp8_v3.py, exp9_classical_unsupervised.py |